

MANUAL TÉCNICO

Tienda de ropa online

React + Spring Boot + PostgreSQL

ASIGNATURA

Arquitecturas Empresariales

Docente: Mauricio Balaguera

Grupo: C

INTEGRANTES

Gisell Alexandra Anaya Santafe

Melanie Yohana Carrillo Parra

Junior Stiven Leal Barragan

Cristian David Mojica Diaz

Nicolle Fabiana Parada Gafaro

Juan Francisco Villamizar Peña

UNIVERSIDAD DE PAMPLONA

Ingeniería de Sistemas

Pamplona, 2026

Tabla de contenido

- 1. Introducción**
- 2. Objetivo del manual técnico**
- 3. Alcance del sistema**
- 4. Tecnologías utilizadas**
- 5. Arquitectura general**
- 6. Estructura del repositorio**
- 7. Requisitos de instalación**
- 8. Configuración y ejecución**
- 9. Base de datos**
- 10. Backend**
- 11. Frontend**
- 12. Seguridad y roles**
- 13. Servicios API REST**
- 14. Flujos principales del sistema**

1. Introducción

El presente manual técnico describe la estructura, tecnologías, configuración, base de datos, servicios y funcionamiento interno de la página web de tienda de ropa online. El sistema fue organizado bajo una arquitectura cliente-servidor, donde el frontend se encarga de la interfaz visual, el backend procesa la lógica del negocio y PostgreSQL almacena la información del sistema.

Este documento está dirigido a desarrolladores, administradores técnicos, docentes o integrantes del equipo que necesiten instalar, ejecutar, revisar o continuar el desarrollo del proyecto.

2. Objetivo del manual técnico

- Explicar de manera organizada cómo está construido técnicamente el sistema de tienda de ropa online, incluyendo su arquitectura, dependencias, estructura de carpetas, configuración, módulos, endpoints, base de datos, seguridad, pruebas y recomendaciones para mantenimiento.

3. Alcance del sistema

La aplicación permite gestionar una tienda de ropa online con autenticación por roles. Se identifican módulos para cliente, administrador y superadministrador.

- Cliente: registro, inicio de sesión, consulta de catálogo, carrito, checkout y compra.
- Administrador: consulta y gestión de productos, inventario, pedidos, ventas, pagos y datos paramétricos.
- Superadministrador: gestión de usuarios, roles, estados de usuario, auditorías y reportes generales.

4. Tecnologías utilizadas

Capa	Tecnología	Uso dentro del proyecto
Frontend	React 19 + Vite	Construcción de la interfaz web, pantallas y consumo de servicios REST.
Backend	Spring Boot 3.5.14	Exposición de la API REST, lógica del negocio, seguridad y conexión con la base de datos.
Lenguaje backend	Java 17	Lenguaje principal del backend.
Base de datos	PostgreSQL	Almacenamiento relacional, funciones, procedimientos, vistas, índices y triggers.
Persistencia	Spring Data JPA + Hibernate	Mapeo de entidades Java con tablas PostgreSQL.
Consultas administrativas	JdbcTemplate	Consulta de vistas y ejecución de procedimientos almacenados.
Seguridad	Spring Security + BCrypt	Control de acceso por roles y cifrado de contraseñas.
Estilos frontend	CSS modular por pantalla	Archivos globales y específicos para catálogo, carrito, checkout, autenticación y paneles.
Gestor frontend	npm	Instalación de dependencias y ejecución del frontend.
Gestor backend	Maven / Maven Wrapper	Compilación y ejecución del backend.

5. Arquitectura general

La arquitectura implementada separa la aplicación en tres capas principales. React no se conecta directamente con PostgreSQL; en su lugar, consume endpoints del backend. Spring Boot recibe las peticiones, aplica validaciones, consulta la base de datos y retorna respuestas en formato JSON.

```

React / Vite
  ↓ HTTP / fetch / API REST
Spring Boot
  ↓ JPA, JdbcTemplate, procedimientos y vistas
PostgreSQL
  
```

Esta separación facilita el trabajo por roles dentro del equipo, el mantenimiento del código y la escalabilidad del sistema.

6. Estructura del repositorio

El repositorio está organizado en cuatro carpetas principales:

▼ TIENDA-ROPA-ONLINE

- > backend
- > database
- > docs
- > frontend

TIENDA-ROPA-ONLINE/

- ├─ backend/ Código del servidor desarrollado con Spring Boot
- ├─ database/ Scripts SQL de la base de datos PostgreSQL
- ├─ docs/ Documentación y evidencias del proyecto
- └─ frontend/ Código de la interfaz web desarrollada con React

6.1 Estructura del backend

- ▼ backend
 - > .mvn\wrapper
 - ▼ src
 - ▼ main
 - ▼ java\com\tienda\backend
 - > config
 - > controller
 - > dto
 - > exception
 - > model
 - > repository
 - > security
 - > service
 - 🔴 BackendApplication.java

```

backend/src/main/java/com/tienda/backend/
├── config/           Configuraciones generales
├── controller/       Controladores REST
├── dto/              Objetos de transferencia de datos
├── exception/        Manejo global de errores
├── model/            Entidades JPA
├── repository/       Acceso a datos con JPA y JdbcTemplate
├── security/         Filtro, token y configuración de seguridad
└── service/         Lógica del negocio

```

6.2 Estructura del frontend

```

▼ frontend
  ▼ tienda-ropa-web
    > public
    ▼ src
      > assets
      > components
      > context
      > data
      > pages
      > routes
      > services
      > styles

```

```

frontend/tienda-ropa-web/src/
├── assets/           Imágenes del proyecto
├── components/       Componentes reutilizables
├── context/          Contextos de autenticación y carrito
├── data/             Datos mock de productos
├── pages/            Pantallas principales
├── routes/           Configuración de rutas y navegación de la aplicación
├── services/         Consumo de API REST
└── styles/          Archivos CSS

```

6.3 Estructura de base de datos

```

▼ database
  01_modelo_general.sql
  02_datos_parametricos_catalogo.sql
  03_ubicaciones.sql
  04_validaciones.sql
  05_productos_inventario_inicial.sql
  06_triggers_auditoria.sql
  07_vistas.sql
  08_indices.sql
  09_permisos_roles.sql
  10_pruebas.sql

```

Elemento	Cantidad identificada
Archivos Java en backend	72
Archivos JS/JSX en frontend	20
Scripts SQL	10
Tablas principales	16
Tablas de auditoría	6

7. Requisitos de instalación

Requisito	Versión/observación
Sistema operativo	Windows, Linux o macOS. En el equipo del proyecto puede trabajarse desde Windows con VS Code, PowerShell o CMD.
Java JDK	Java 17 o superior compatible con Spring Boot.
Maven	Se puede usar Maven instalado o el Maven Wrapper incluido en backend/mvnw y backend/mvnw.cmd.
Node.js y npm	Necesarios para instalar dependencias y ejecutar el frontend React + Vite.
PostgreSQL	Motor de base de datos relacional usado por el proyecto.
Git	Control de versiones y manejo del repositorio.
Postman o similar	Herramienta recomendada para probar endpoints REST.

8. Configuración y ejecución

8.1 Creación de la base de datos

Crear una base de datos llamada tienda_ropa_online en PostgreSQL:

```
CREATE DATABASE tienda_ropa_online;
```

Después se deben ejecutar los scripts SQL en el siguiente orden:

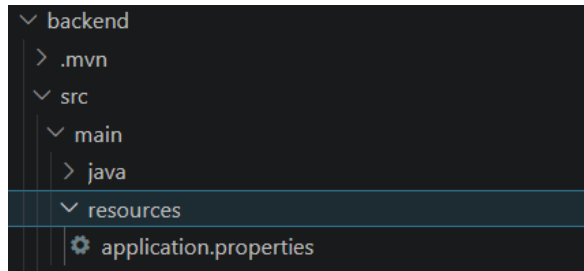
- 01_modelo_general.sql
- 02_datos_parametricos_catalogo.sql
- 03_ubicaciones.sql
- 04_validaciones.sql
- 05_productos_inventario_inicial.sql
- 06_triggers_auditoria.sql
- 07_vistas.sql
- 08_indices.sql
- 09_permisos_rols.sql
- 10_pruebas.sql

Para ejecutar cada script en pgAdmin, se debe seleccionar la base de datos tienda_ropa_online, abrir la opción Query Tool, copiar el contenido del archivo SQL correspondiente y presionar el botón Execute o la tecla F5. Es importante ejecutar los archivos en el orden indicado, ya que algunos scripts dependen de tablas, funciones, vistas o datos creados previamente en archivos anteriores.

8.2 Configuración del backend

El archivo principal de configuración se encuentra en:

backend/src/main/resources/application.properties



La conexión a PostgreSQL está preparada para usar variables de entorno:

```

application.properties
backend > src > main > resources > application.properties
1  spring.application.name=backend
2
3  spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/tienda_ropa_online}
4  spring.datasource.username=${DB_USERNAME:postgres}
5  spring.datasource.password=${DB_PASSWORD:}
6
7  spring.jpa.hibernate.ddl-auto=none
8  spring.jpa.show-sql=true
9  spring.jpa.properties.hibernate.format_sql=true
10 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

```

En Windows se pueden definir las variables antes de ejecutar el backend:

```

PS C:\Users\Usuario\tienda-ropa-online> $env:DB_URL="jdbc:postgresql://localhost:5432/tienda_ropa_online"
PS C:\Users\Usuario\tienda-ropa-online> $env:DB_USERNAME="postgres"
PS C:\Users\Usuario\tienda-ropa-online> $env:DB_PASSWORD="tu_contrasena"

```

Para ejecutar el backend:

```

PS C:\Users\Usuario\tienda-ropa-online> cd backend
PS C:\Users\Usuario\tienda-ropa-online\backend> .\mvnw.cmd spring-boot:run

```

Por defecto, Spring Boot se ejecuta en el puerto 8080.

8.3 Configuración del frontend

El frontend está ubicado en frontend/tienda-ropa-web. Para instalar y ejecutar:

```

PS C:\Users\Usuario\tienda-ropa-online> cd .\frontend\tienda-ropa-web
PS C:\Users\Usuario\tienda-ropa-online\frontend\tienda-ropa-web> npm install

up to date, audited 136 packages in 792ms

31 packages are looking for funding
  run 'npm fund' for details

1 high severity vulnerability

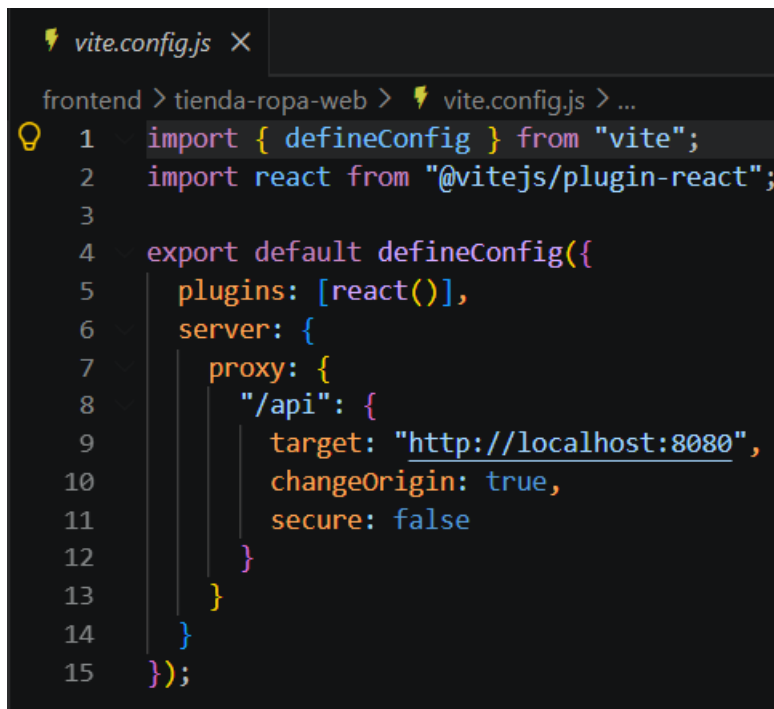
To address all issues, run:
  npm audit fix

Run 'npm audit' for details.
PS C:\Users\Usuario\tienda-ropa-online\frontend\tienda-ropa-web> npm run dev

```

La aplicación Vite queda disponible normalmente en <http://localhost:5173>.

El archivo vite.config.js incluye un proxy para redirigir las peticiones /api hacia el backend:



```

vite.config.js
frontend > tienda-ropa-web > vite.config.js > ...
1 import { defineConfig } from "vite";
2 import react from "@vitejs/plugin-react";
3
4 export default defineConfig({
5   plugins: [react()],
6   server: {
7     proxy: {
8       "/api": {
9         target: "http://localhost:8080",
10        changeOrigin: true,
11        secure: false
12      }
13    }
14  }
15 });

```

9. Base de datos

La base de datos se diseñó en PostgreSQL. El script 01_modelo_general.sql define las tablas principales y sus relaciones; los demás scripts agregan datos paramétricos, validaciones, procedimientos, triggers, vistas, índices, permisos y pruebas.

9.1 Tablas principales

Tabla	Propósito
roles	Almacena los roles CLIENTE, ADMIN y SUPERADMIN.
personas	Registra usuarios del sistema con datos personales, correo, contraseña cifrada, estado activo y rol.

Tabla	Propósito
departamentos / municipios	Almacenan información geográfica para direcciones.
direcciones / personas_direcciones	Relacionan usuarios con direcciones.
categorias	Clasificación principal del producto, por ejemplo Superior, Inferior o Calzado.
estilos	Estilo del producto, por ejemplo Deportivo, Casual, Urban o Elegante.
tallas	Tallas de ropa y calzado.
colores	Colores disponibles para inventario.
productos	Información general del producto: nombre, precio, imagen, estado, categoría y estilo.
inventarios	Controla stock por combinación producto, talla y color.
ventas	Registra la venta realizada por una persona.
detalle_ventas	Registra los productos vendidos, cantidades y precio unitario.
metodos_pago	Métodos disponibles: tarjeta de crédito, tarjeta de débito y PSE.
pagos	Registra el pago asociado a una venta.

9.2 Tablas de auditoría

El script 06_triggers_auditoria.sql crea tablas y triggers para registrar cambios en entidades importantes:

- aud_personas
- aud_productos
- aud_inventarios
- aud_ventas
- aud_detalle_ventas
- aud_pagos

9.3 Funciones y procedimientos principales

Objeto	Tipo	Descripción
fn_validar_correo	Función	Valida que el correo no esté vacío y cumpla formato de correo electrónico.
fn_validar_telefono	Función	Valida número telefónico colombiano que inicia por 3 y tiene 10 dígitos.
fn_validar_stock	Función	Evita valores nulos o negativos en stock.
fn_estado_producto	Función	Retorna AGOTADO si el stock es menor o igual a cero; de lo contrario retorna DISPONIBLE.
registrar_inventario	Procedimiento	Registra o aumenta inventario por producto, talla y color.
aumentar_stock	Procedimiento	Incrementa la cantidad disponible en inventario.
cambiar_estado_persona	Procedimiento	Activa o desactiva usuarios.

Objeto	Tipo	Descripción
registrar_cliente	Procedimiento	Registra personas con rol CLIENTE desde base de datos.
registrar_usuario_admin	Procedimiento	Registra usuarios administrativos.
realizar_compra_carrito	Función	Registra una venta con detalle y pago a partir de un JSON de productos.
registrar_producto	Procedimiento	Crea productos desde el módulo administrativo.
editar_producto	Procedimiento	Actualiza datos del producto.
cambiar_estado_producto	Procedimiento	Activa o desactiva productos.
actualizar_inventario	Procedimiento	Actualiza directamente el stock de un inventario.

9.4 Vistas principales

Vista	Uso
vw_catalogo_productos	Consulta general de catálogo con stock total, tallas y colores disponibles.
vw_catalogo_productos_detalle	Consulta detallada por talla, color e inventario.
vw_admin_productos	Listado de productos para el panel administrador.
vw_admin_inventario	Inventario completo con producto, talla, color y estado.
vw_resumen_ventas	Resumen de ventas con cliente, método de pago y monto.
vw_detalle_ventas_admin	Detalle de ventas para administración.
vw_pedidos_cliente	Pedidos agrupados por cliente.
vw_detalle_pedido_cliente	Detalle de productos de un pedido específico.
vw_usuarios_sistema	Listado de usuarios con rol y estado.
vw_auditoria_general	Unificación de registros de auditoría.
mv_resumen_ventas_productos	Vista materializada para resumen de ventas por producto.

9.5 Datos paramétricos iniciales

El script 02_datos_parametricos_catalogo.sql inserta valores base para categorías, estilos, métodos de pago, tallas, colores y roles.

Grupo	Valores destacados
Categorías	Superior, Inferior, Calzado
Estilos	Deportivo, Elegante, Urban, Casual
Métodos de pago	Tarjeta de crédito, Tarjeta de débito, PSE
Roles	CLIENTE, ADMIN, SUPERADMIN
Tallas	XS, S, M, L, XL y tallas de calzado 35 a 44

Grupo	Valores destacados
Colores	Negro, Blanco, Rojo, Azul, Verde, Gris, Café, Beige, Oliva, entre otros

10. Backend

El backend está construido en Spring Boot y se encuentra en la carpeta backend. Su clase principal es BackendApplication.java. El sistema expone servicios REST, aplica seguridad por roles, maneja validaciones y se comunica con PostgreSQL mediante JPA, EntityManager y JdbcTemplate.

10.1 Paquetes del backend

Paquete	Responsabilidad
controller	Recibe peticiones HTTP y retorna respuestas JSON.
service	Contiene la lógica del negocio.
repository	Comunicación con la base de datos mediante JPA y JdbcTemplate.
model	Entidades que representan tablas de PostgreSQL.
dto	Objetos usados para entrada y salida de datos.
security	Configuración de seguridad, generación de token y filtro de autenticación.
exception	Manejo global de errores y validaciones.

10.2 Controladores principales

Controlador	Ruta base	Función
AuthController	/api/auth	Registro, login, consulta de usuario autenticado y roles.
ProductoController	/api/productos	Consulta pública del catálogo.
InventarioController	/api/inventarios/validar	Validación de stock.
CompraController	/api/compras	Registro de compras de clientes autenticados.
ClienteController	/api/cliente	Perfil, direcciones y pedidos del cliente.
AdminController	/api/admin	Panel administrativo de productos, inventario, ventas, pagos y pedidos.
UsuarioController	/api/usuarios	Gestión de usuarios por superadministrador.
AuditoriaController	/api/auditorias	Consulta de auditoría general o por tabla.

Controlador	Ruta base	Función
ReporteController	/api/reportes	Reportes generales y ventas por producto.

10.3 Entidades principales

Entidad Java	Tabla relacionada
Rol	roles
Persona	personas
Categoría	categorías
Estilo	estilos
Talla	tallas
Color	colores
Producto	productos
Inventario	inventarios

10.4 DTO principales

DTO	Uso
RegistroRequestDTO	Datos para registrar cliente.
LoginRequestDTO	Credenciales para iniciar sesión.
AuthResponseDTO	Respuesta de autenticación con datos de usuario, rol, estado y token.
ProductoDTO	Información del producto enviada al catálogo.
InventarioDTO	Existencias por inventario, talla, color y stock.
CompraRequestDTO	Datos de compra: método de pago y detalles.
CompraDetalladoDTO	Inventario y cantidad de cada producto comprado.
CompraResponseDTO	Resultado de la compra y número de venta.
UsuarioCreateDTO	Datos para crear usuario administrativo.
ApiResponse	Respuesta genérica con éxito, mensaje y datos.

10.5 Manejo de errores

El proyecto incluye `GlobalExceptionHandler`, encargado de capturar errores de validación, argumentos inválidos y excepciones de ejecución. Las respuestas devuelven un objeto JSON con los campos `exit` y `mensaje`.

```
{
  "exit": false,
```

```
"mensaje": "El correo no tiene un formato válido"
}
```

11. Frontend

El frontend está desarrollado con React y Vite. La navegación principal se controla desde App.jsx mediante un estado llamado vistaActual. El acceso a las pantallas depende de la sesión y del rol del usuario autenticado.

11.1 Pantallas principales

Pantalla	Archivo	Función
Login	LoginPage.jsx	Permite iniciar sesión con correo y contraseña.
Registro	RegisterPage.jsx	Permite registrar clientes.
Catálogo	CatalogPage.jsx	Consulta productos desde el backend y los muestra en tarjetas.
Carrito	CartPage.jsx	Muestra productos agregados, cantidades, subtotal, envío y total.
Checkout	CheckoutPage.jsx	Valida datos de envío, método de pago, stock y registra la compra.
Panel administrador	AdminDashboardPage.jsx	Gestión de productos, inventario, ventas, pagos y pedidos.
Panel superadministrador	SuperAdminDashboardPage.jsx	Gestión de usuarios, roles, estados, auditorías y reportes.

11.2 Componentes y contextos

Archivo	Descripción
Navbar.jsx	Barra de navegación para cliente autenticado.
ProductCard.jsx	Tarjeta de producto con selección de talla, color, cantidad y botón de agregar.
AuthContext.jsx	Manejo de sesión, token, usuario y rol. Guarda información en localStorage con clave shopnmg_auth.
CartContext.jsx	Manejo del carrito. Guarda productos en localStorage con clave tienda_carrito.

11.3 Servicios frontend

Servicio	Endpoints que consume
authService.js	/api/auth/login, /api/auth/registro, /api/auth/me
productService.js	/api/productos

Servicio	Endpoints que consume
productosService.js	/api/productos para validar stock del carrito con datos actualizados.
checkoutService.js	/api/compras
adminService.js	/api/admin/productos, /api/admin/inventario, /api/admin/pedidos, /api/admin/ventas, /api/admin/pagos, catálogos paramétricos.
superAdminService.js	/api/usuarios, /api/auditorias, /api/reportes/general

11.4 Flujo de navegación por rol

```

Login correcto
├── rol CLIENTE      → catálogo
├── rol ADMIN        → panel administrador
└── rol SUPERADMIN  → panel superadministrador

```

12. Seguridad y roles

La seguridad está configurada con Spring Security. El sistema desactiva formularios de login y HTTP Basic, usa un filtro propio AuthTokenFilter y controla el acceso por roles. Las contraseñas registradas desde el backend se cifran con BCryptPasswordEncoder.

Rol	Permisos principales
CLIENTE	Catálogo, carrito, checkout, compras, perfil, direcciones y pedidos propios.
ADMIN	Gestión administrativa de productos, inventario, ventas, pedidos, pagos y reportes.
SUPERADMIN	Permisos administrativos y gestión de usuarios, roles, estados y auditorías.

12.1 Token de autenticación

El servicio AuthTokenService genera un token codificado en Base64 URL-safe con el identificador del usuario, rol y timestamp. El frontend lo envía en el encabezado Authorization con el formato Bearer token.

```
Authorization: Bearer <token>
```

13. Servicios API REST

13.1 Endpoints públicos

Método	Endpoint	Descripción
POST	/api/auth/registro	Registra un cliente.
POST	/api/auth/login	Inicia sesión y retorna datos de usuario y token.
GET	/api/auth/roles	Consulta roles existentes.

Método	Endpoint	Descripción
GET	/api/productos	Consulta catálogo de productos activos.
GET	/api/inventarios/validar	Valida stock mediante productold, tallald, colorId y cantidad.

13.2 Endpoints de cliente

Método	Endpoint	Descripción
GET	/api/cliente/perfil	Obtiene perfil del cliente autenticado.
GET	/api/cliente/direcciones	Lista direcciones del cliente.
POST	/api/cliente/direcciones	Registra dirección del cliente.
PUT	/api/cliente/direcciones/{direccionId}	Actualiza dirección.
GET	/api/cliente/pedidos	Lista pedidos del cliente.
GET	/api/cliente/pedidos/{pedidoId}	Consulta detalle de un pedido.
POST	/api/compras	Registra una compra del cliente autenticado.

13.3 Endpoints de administrador

Método	Endpoint	Descripción
GET	/api/admin/productos	Lista productos para administración.
GET	/api/admin/inventario	Lista inventario administrativo.
GET	/api/admin/resumen	Obtiene resumen administrativo.
GET	/api/admin/ventas	Lista ventas.
GET	/api/admin/pedidos	Lista pedidos.
GET	/api/admin/pagos	Lista pagos.
POST	/api/admin/productos	Registra producto.
PUT	/api/admin/productos/{id}	Edita producto.
PATCH	/api/admin/productos/{id}/estado	Activa o desactiva producto.
POST	/api/admin/inventario	Registra inventario.
PUT	/api/admin/inventario/{id}	Actualiza stock.
GET	/api/admin/categorias	Lista categorías.
GET	/api/admin/estilos	Lista estilos.
GET	/api/admin/tallas	Lista tallas.
GET	/api/admin/colores	Lista colores.
GET	/api/reportes/general	Reporte general.

Método	Endpoint	Descripción
GET	/api/reportes/ventas-productos	Ventas por producto.

13.4 Endpoints de superadministrador

Método	Endpoint	Descripción
POST	/api/usuarios	Crea usuario administrativo.
GET	/api/usuarios	Lista usuarios del sistema.
GET	/api/usuarios/{id}	Consulta usuario por id.
PATCH	/api/usuarios/{id}/estado	Activa o desactiva usuario.
PATCH	/api/usuarios/{id}/rol	Cambia rol del usuario.
GET	/api/auditorias	Lista auditoría general.
GET	/api/auditorias/tabla/{tabla}	Lista auditoría filtrada por tabla.

14. Flujos principales del sistema

14.1 Flujo de autenticación

- El usuario ingresa correo y contraseña en LoginPage.jsx.
- El frontend envía la petición a /api/auth/login usando authService.js.
- AuthService busca el usuario por correo y valida la contraseña con BCrypt.
- Si el usuario existe, está activo y la contraseña coincide, se genera un token.
- El frontend guarda usuario y token en localStorage.
- App.jsx redirige según el rol: CLIENTE, ADMIN o SUPERADMIN.

14.2 Flujo de catálogo

- CatalogPage.jsx solicita productos a /api/productos.
- ProductoController llama a ProductoService.
- ProductoService consulta productos activos y sus inventarios.
- El backend retorna ProductoDTO con existencias por talla, color y stock.
- El frontend muestra los productos y bloquea combinaciones sin stock.

14.3 Flujo de compra

- El cliente agrega productos al carrito usando ProductCard.jsx y CartContext.jsx.
- CheckoutPage.jsx valida datos de envío y método de pago.
- productosService.js consulta el catálogo actualizado para validar stock antes de comprar.
- checkoutService.js envía la compra a POST /api/compras con token Bearer.
- CompraController obtiene el cliente desde el token y llama a CompraServiceImp.
- CompraServiceImp arma un JSON con inventario y cantidad.
- PostgreSQL ejecuta realizar_compra_carrito, registra venta, detalle, pago y descuenta stock.

14.4 Flujo administrativo

29. El administrador ingresa al panel AdminDashboardPage.jsx.
30. El frontend consulta productos, inventario, ventas, pedidos, pagos y catálogos paramétricos.
31. El administrador puede crear productos, editarlos, cambiar su estado, registrar inventario y actualizar stock.
32. El backend usa AdminJdbcRepository para consultar vistas y ejecutar procedimientos almacenados.

14.5 Flujo superadministrador

33. El superadministrador ingresa al panel SuperAdminDashboardPage.jsx.
34. Puede crear usuarios administrativos, activar/desactivar usuarios y cambiar roles.
35. También puede consultar auditorías y reportes generales del sistema.
36. Las operaciones se realizan mediante /api/usuarios, /api/auditorias y /api/reportes/general.